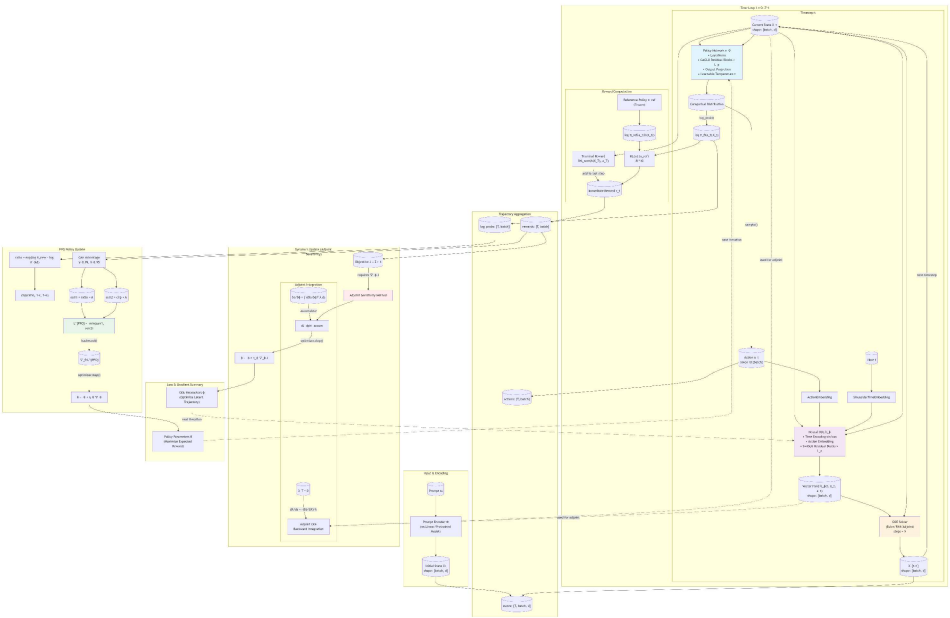

Figure 1. Overall computational graph illustrating the forward pass (solid arrows) and gradient flow (dashed arrows) for joint optimization of policy network π_θ and neural ODE dynamics b_ϕ. The graph consists of three main phases: (1) trajectory generation via ODE solving with per-step KL regularization, (2) policy update using PPO with GAE advantage estimation, and (3) dynamics update via adjoint sensitivity method. Detailed specifications of the involved modules and corresponding functions are provided in the subsequent tables.

Attribute	**Description**
Class Name	`SinusoidalTimeEmbedding`
Description	Sinusoidal time encoding module that maps scalar time values to high-dimensional vectors for time-conditioned modeling in neural ODEs
Function	**Parameters & Return**
`__init__()`	`d_model: int` - Output embedding dimension Return: None Initializes sinusoidal encoder with specified output dimension
`forward()`	`t: torch.Tensor` - Time values, shape `()` (scalar) or `(batch,)` Return: `torch.Tensor` of shape `(batch, d_model)` Encodes time using sinusoidal/cosine functions at different frequencies

Table 1. `SinusoidalTimeEmbedding` - Sinusoidal Time Encoding

Attribute	**Description**
Class Name	`ActionEmbedding`
Description	Token embedding module that maps discrete token IDs to continuous vector space for conditioning the ODE vector field
Function	**Parameters & Return**
`__init__()`	`vocab_size: int` - Vocabulary size `embed_dim: int` - Embedding dimension Return: None Initializes embedding layer with `nn.Embedding()`
`forward()`	`a: torch.Tensor` - Token IDs, shape `(batch,)` or `(batch, seq_len)` Return: `torch.Tensor` of shape `(batch, embed_dim)` or `(batch, seq_len, embed_dim)` Maps discrete token IDs to continuous embedding vectors

Table 2. `ActionEmbedding` - Action (Token) Embedding

Attribute	Description
Class Name	`NeuralODEDynamics`
Description	Parameterized vector field that defines continuous-time evolution of latent states: $\mathrm{d}X_s = b_\phi(s, X_s, a_t)\mathrm{d}s$
Function	**Parameters & Return**
`__init__()`	`d_state: int` - Latent state dimension `vocab_size: int` - Vocabulary size `num_layers: int` - Number of residual blocks `expansion_ratio: int` - FFN expansion factor Return: None Initializes network with time encoding, action embedding, SwiGLU residual blocks, and output projection

`forward()`	`s: torch.Tensor` - Time values, shape `(batch,)`
	`X: torch.Tensor` - Current state, shape `(batch, d_state)`
	`a: torch.Tensor` - Action token, shape `(batch,)`
	Return: `torch.Tensor` of shape `(batch, d_state)`
	Computes vector field $b_\phi(s, X_s, a_t) = \mathrm{d}X_s/\mathrm{d}s$

Table 3. `NeuralODEDynamics` - Neural ODE Dynamics $b_\phi(s, X_s, a_t)$

Attribute	Description
Class Name	`PolicyNetwork`
Description	Stochastic policy that outputs categorical distribution over actions (tokens) given latent state
Function	**Parameters & Return**
`__init__()`	`d_state: int` - Latent state dimension `vocab_size: int` - Vocabulary size `num_layers: int` - Number of residual blocks `expansion_ratio: int` - FFN expansion factor Return: None Initializes policy with GeGLU residual blocks, learnable temperature parameter, and output projection
`forward()`	`X: torch.Tensor` - Latent state, shape `(batch, d_state)` Return: `torch.distributions.Categorical` Returns categorical distribution for action sampling and log probability computation

Table 4. `PolicyNetwork`- Policy Network $\pi_\theta(\cdot \mid X_t)$

Attribute	**Description**
Class Name	`TextGenerationEnv`
Description	Environment simulator that encapsulates prompt encoding and ODE-based trajectory generation
Function	**Parameters & Return**
`__init__()`	`prompt_encoder: nn.Module` - Prompt encoder `dynamics: NeuralODEDynamics` - ODE dynamics network `max_len: int` - Maximum generation length `ode_steps_per_token: int` - ODE solver steps per token interval Return: None Initializes environment with encoder, dynamics model, and hyperparameters
`reset()`	`prompt: List[str]` - Input prompts Return: `torch.Tensor` of shape `(batch, d_state)` Encodes prompts to initial latent state X_0
`step()`	`X: torch.Tensor` - Current state `a: torch.Tensor` - Action token `t: int` - Current timestep Return: `torch.Tensor` - Next state X_{t+1} Executes action and evolves state via ODE solver over interval $(t, t+1]$

Table 5. `TextGenerationEnv` - Text Generation Environment

Attribute	**Description**
Class Name	`ContinuousLatentRLTrainer`
Description	Main training orchestrator integrating PPO policy updates and adjoint sensitivity dynamics updates
Function	**Parameters & Return**
`__init__()`	Multiple hyperparameters for RL training Return: None Initializes trainer, creates optimizers, and stores all components
`compute_kl_divergence()`	`log_probs_student: torch.Tensor` - Student policy log probs `log_probs_ref: torch.Tensor` - Reference policy log probs Return: `torch.Tensor` - KL divergence value Computes KL divergence for distillation regularization: $\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}})$
`compute_gae()`	`rewards: torch.Tensor` - Reward sequence `values: torch.Tensor` - Value function estimates `dones: torch.Tensor` - Termination flags Return: `torch.Tensor` - GAE advantage estimates Computes Generalized Advantage Estimation: $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$
`collect_trajectory()`	`prompt: List[str]` - Input prompt Return: `dict` with keys: states, actions, log_probs, rewards Samples complete trajectory with per-step KL rewards and terminal reward
`update_policy()`	`traj: dict` - Trajectory data Return: `float` - PPO loss value Updates policy using PPO with clipped surrogate objective
`update_dynamics()`	`traj: dict` - Trajectory data Return: `float` - Loss value Updates ODE parameters using adjoint sensitivity method (demonstrative implementation)
`train_step()`	`prompts: List[str]` - Input prompts Return: `Tuple[float, float]` - (policy loss, dynamics loss) Executes single training iteration: trajectory collection → policy update → dynamics update

Table 6. `ContinuousLatentRLTrainer` - Main Training Orchestrator

Class Name	Description
`GeGLU`	GeGLU activation using GELU gating: $\text{GeGLU}(x) = W_3(\text{GELU}(W_1 x) \odot W_2 x)$
`SwiGLU`	SwiGLU activation using SiLU (Swish) gating: $\text{SwiGLU}(x) = W_3(\text{SiLU}(W_1 x) \odot W_2 x)$
Function	**Parameters & Return**
`__init__()`	`dim: int` - Input dimension `expansion_ratio: int` - Hidden dimension = $dim \times expansion_ratio$ Return: None Initializes three linear layers for gating mechanism
`forward()`	`x: torch.Tensor` - Input tensor Return: `torch.Tensor` - Activated output Applies gated linear unit with specified activation function

Table 7. `GeGLU` and `SwiGLU` Activation Modules